

Utilities

The time you enjoy wasting is not wasted time.
– Bertrand Russell

- Introduction
- Resource Management
 - `unique_ptr` and `shared_ptr`
- Specialized Containers
 - `array`; `bitset`; `pair` and `tuple`
- Time
- Function Adaptors
 - `bind()`; `mem_fn()`; `function`
- Type Functions
 - `iterator_traits`; Type Predicates
- Advice

11.1 Introduction

Not all standard-library components come as part of obviously labeled facilities, such as “containers” or “I/O.” This section gives a few examples of small, widely useful components. The point here is that a function or a type need not be complicated or closely tied to a mass of other functions and types to be useful. Such library components mostly act as building blocks for more powerful library facilities, including other components of the standard library.

11.2 Resource Management

One of the key tasks of any nontrivial program is to manage resources. A resource is something that must be acquired and later (explicitly or implicitly) released. Examples are memory, locks,

sockets, thread handles, and file handles. For a long-running program, failing to release a resource in a timely manner (“a leak”) can cause serious performance degradation and possibly even a miserable crash. Even for short programs, a leak can become an embarrassment, say by a resource shortage increasing the run time by orders of magnitude.

The standard library components are designed not to leak resources. To do this, they rely on the basic language support for resource management using constructor/destructor pairs to ensure that a resource doesn’t outlive an object responsible for it. The use of a constructor/destructor pair in **Vector** to manage the lifetime of its elements is an example (§4.2.2) and all standard-library containers are implemented in similar ways. Importantly, this approach interacts correctly with error handling using exceptions. For example, the technique is used for the standard-library lock classes:

```
mutex m; // used to protect access to shared data
// ...
void f()
{
    unique_lock<mutex> lck {m}; // acquire the mutex m
    // ... manipulate shared data ...
}
```

A **thread** will not proceed until **lck**’s constructor has acquired its **mutex**, **m** (§13.5). The corresponding destructor releases the resource. So, in this example, **unique_lock**’s destructor releases the **mutex** when the thread of control leaves **f()** (through a **return**, by “falling off the end of the function,” or through an exception throw).

This is an application of the “Resource Acquisition Is Initialization” technique (RAII; §4.2.2). RAI is fundamental to the idiomatic handling of resources in C++. Containers (such as **vector** and **map**), **string**, and **iostream** manage their resources (such as file handles and buffers) similarly.

11.2.1 **unique_ptr** and **shared_ptr**

The examples so far take care of objects defined in a scope, releasing the resources they acquire at the exit from the scope, but what about objects allocated on the free store? In **<memory>**, the standard library provides two “smart pointers” to help manage objects on the free store:

- [1] **unique_ptr** to represent unique ownership
- [2] **shared_ptr** to represent shared ownership

The most basic use of these “smart pointers” is to prevent memory leaks caused by careless programming. For example:

```
void f(int i, int j)    // X* vs. unique_ptr<X>
{
    X* p = new X;        // allocate a new X
    unique_ptr<X> sp {new X}; // allocate a new X and give its pointer to unique_ptr
    // ...
    if (i<99) throw Z{};   // may throw an exception
    if (j<77) return;      // may return "early"
    // ...
    p->do_something();     // may throw an exception
    sp->do_something();    // may throw an exception
}
```

```

// ...
delete p;                // destroy *p
}

```

Here, we “forgot” to delete `p` if `i<99` or if `j<77`. On the other hand, `unique_ptr` ensures that its object is properly destroyed whichever way we exit `f()` (by throwing an exception, by executing `return`, or by “falling off the end”). Ironically, we could have solved the problem simply by *not* using a pointer and *not* using `new`:

```

void f(int i, int j)    // use a local variable
{
    X x;
    // ...
}

```

Unfortunately, overuse of `new` (and of pointers and references) seems to be an increasing problem.

However, when you really need the semantics of pointers, `unique_ptr` is a very lightweight mechanism with no space or time overhead compared to correct use of a built-in pointer. Its further uses include passing free-store allocated objects in and out of functions:

```

unique_ptr<X> make_X(int i)
    // make an X and immediately give it to a unique_ptr
{
    // ... check i, etc. ...
    return unique_ptr<X>{new X{i}};
}

```

A `unique_ptr` is a handle to an individual object (or an array) in much the same way that a `vector` is a handle to a sequence of objects. Both control the lifetime of other objects (using RAII) and both rely on move semantics to make `return` simple and efficient.

The `shared_ptr` is similar to `unique_ptr` except that `shared_ptr`s are copied rather than moved. The `shared_ptr`s for an object share ownership of an object and that object is destroyed when the last of its `shared_ptr`s is destroyed. For example:

```

void f(shared_ptr<fstream>);
void g(shared_ptr<fstream>);

void user(const string& name, ios_base::openmode mode)
{
    shared_ptr<fstream> fp {new fstream(name,mode)};
    if (!*fp)                // make sure the file was properly opened
        throw No_file{};

    f(fp);
    g(fp);
    // ...
}

```

Now, the file opened by `fp`’s constructor will be closed by the last function to (explicitly or implicitly) destroy a copy of `fp`. Note that `f()` or `g()` may spawn a task holding a copy of `fp` or in some other way store a copy that outlives `user()`. Thus, `shared_ptr` provides a form of garbage collection

that respects the destructor-based resource management of the memory-managed objects. This is neither cost free nor exorbitantly expensive, but it does make the lifetime of the shared object hard to predict. Use `shared_ptr` only if you actually need shared ownership.

Creating an object on the free store and then passing a pointer to it to a smart pointer is logically a bit odd and can be verbose. To compensate, the standard library (in `<memory>`) provides a function `make_shared()`. For example:

```
struct S {
    int i;
    string s;
    double d;
    // ...
};

shared_ptr<S> p1 {new S {1,"Ankh Morpork",4.65}};

auto p2 = make_shared<S>(2,"Oz",7.62);
```

Now, `p2` is a `shared_ptr<S>` pointing to an object of type `S` allocated on the free store, containing `{1,string{"Ankh Morpork"},4.65}`.

Currently, there is no standard-library `make_unique()` similar to `make_shared()` and `make_pair()` (§11.3.3). However, it is easily defined:

```
template<typename T, typename... Args>
unique_ptr<T> make_unique(Args&&... args)
{
    return std::unique_ptr<T>{new T{std::forward<Args>(args)...}};
}
```

No, I don't claim that this definition is trivial to understand, but it is efficient and quite general. The ellipses, `...`, indicate the use of a variadic template (§5.6). We can now write:

```
auto p2 = make_unique<S>(3,"Atlantis",11.3);
```

Given `unique_ptr` and `shared_ptr`, we can implement a complete “no naked `new`” policy (§4.2.2) for many programs. However, these “smart pointers” are still conceptually pointers and therefore only my second choice for resource management – after containers and other types that manage their resources at a higher conceptual level. In particular, `shared_ptr`s do not in themselves provide any rules for which of their owners can read and/or write the shared object. Data races (§13.7) and other forms of confusion are not addressed simply by eliminating the resource management issues.

Where do we use “smart pointers” (such as `unique_ptr`) rather than resource handles with operations designed specifically for the resource (such as `vector` or `thread`)? Unsurprisingly, the answer is “when we need pointer semantics.”

- When we share an object, we need pointers (or references) to refer to the shared object, so a `shared_ptr` becomes the obvious choice (unless there is an obvious single owner).
- When we refer to a polymorphic object, we need a pointer (or a reference) because we don't know the exact type of the object referred to (or even its size), so a `unique_ptr` becomes the obvious choice.

- A shared polymorphic object typically requires `shared_ptrs`.

We do *not* need to use a pointer to return a collection of objects from a function; a container that is a resource handle will do that simply and efficiently (§4.6.2).

11.3 Specialized Containers

The standard library provides several containers that don’t fit perfectly into the STL framework (Chapter 9, Chapter 10). Examples are built-in arrays, `array`, and `string`. I sometimes refer to those as “almost containers,” but that is not quite fair: they hold elements, so they are containers, but each has restrictions or added facilities that make them awkward in the context of the STL. Describing them separately also simplifies the description of the STL.

“Almost Containers”	
<code>T[N]</code>	Built-in array: a fixed-size continuously allocated sequence of <code>N</code> elements of type <code>T</code> ; implicitly converts to a <code>T*</code>
<code>array<T,N></code>	A fixed-size continuously allocated sequence of <code>N</code> elements of type <code>T</code> ; like the built-in array, but with most problems solved
<code>bitset<N></code>	A fixed-size sequence of <code>N</code> bits
<code>vector<bool></code>	A sequence of bits compactly stored in a specialization of <code>vector</code>
<code>pair<T,U></code>	Two elements of types <code>T</code> and <code>U</code>
<code>tuple<T...></code>	A sequence of an arbitrary number of elements of arbitrary types
<code>basic_string<C></code>	A sequence of characters of type <code>C</code> ; provides string operations
<code>valarray<T></code>	An array of numeric values of type <code>T</code> ; provides numeric operations

Why does the standard library provide so many containers? They serve common but different (often overlapping) needs. If the standard library didn’t provide them, many people would have to design and implement their own. For example:

- `pair` and `tuple` are heterogeneous; all other containers are homogeneous (all elements are of the same type).
- `array`, `vector`, and `tuple` elements are contiguously allocated; `forward_list` and `map` are linked structures.
- `bitset` and `vector<bool>` hold bits and access them through proxy objects; all other standard-library containers can hold a variety of types and access elements directly.
- `basic_string` requires its elements to be some form of character and to provide string manipulation, such as concatenation and locale-sensitive operations
- `valarray` requires its elements to be numbers and to provide numerical operations.

All of these containers can be seen as providing specialized services needed by large communities of programmers. No single container could serve all of these needs because some needs are contradictory, for example, “ability to grow” vs. “guaranteed to be allocated in a fixed location,” and “elements do not move when elements are added” vs. “contiguously allocated.” Furthermore, a very general container would imply overhead deemed unacceptable for individual containers.

11.3.1 array

An **array**, defined in `<array>`, is a fixed-size sequence of elements of a given type where the number of elements is specified at compile time. Thus, an **array** can be allocated with its elements on the stack, in an object, or in static storage. The elements are allocated in the scope where the **array** is defined. An **array** is best understood as a built-in array with its size firmly attached, without implicit, potentially surprising conversions to pointer types, and with a few convenience functions provided. There is no overhead (time or space) involved in using an **array** compared to using a built-in array. An **array** does *not* follow the “handle to elements” model of STL containers. Instead, an **array** directly contains its elements.

An **array** can be initialized by an initializer list:

```
array<int,3> a1 = {1,2,3};
```

The number of elements in the initializer must be equal to or less than the number of elements specified for the **array**.

The element count is not optional:

```
array<int> ax = {1,2,3};           // error size not specified
```

The element count must be a constant expression:

```
void f(int n)
{
    array<string,n> aa = {"John's", "Queens' "};    // error: size not a constant expression
    //
}
```

If you need the element count to be a variable, use **vector**.

When necessary, an **array** can be explicitly passed to a C-style function that expects a pointer. For example:

```
void f(int* p, int sz);           // C-style interface

void g()
{
    array<int,10> a;

    f(a,a.size());                // error: no conversion
    f(&a[0],a.size());             // C-style use
    f(a.data(),a.size());          // C-style use

    auto p = find(a.begin(),a.end(),777);    // C++/STL-style use
    // ...
}
```

Why would we use an **array** when **vector** is so much more flexible? Because an **array** is less flexible, it is simpler. Occasionally, there is a significant performance advantage to be had by directly accessing elements allocated on the stack rather than allocating elements on the free store, accessing them indirectly through the **vector** (a handle), and then deallocating them. On the other hand, the stack is a limited resource (especially on some embedded systems), and stack overflow is nasty.

Why would we use an **array** when we could use a built-in array? An **array** knows its size, so it is easy to use with standard-library algorithms, and it can be copied (using `=` or initialization). However, my main reason to prefer **array** is that it saves me from surprising nasty conversions to pointers. Consider:

```
void h()
{
    Circle a1[10];
    array<Circle,10> a2;
    // ...
    Shape* p1 = a1;    // OK: disaster waiting to happen
    Shape* p2 = a2;    // error: no conversion of array<Circle,10> to Shape*
    p1[3].draw();      // disaster
}
```

The “disaster” comment assumes that `sizeof(Shape)<sizeof(Circle)`, so that subscripting a `Circle[]` through a `Shape*` gives a wrong offset. All standard containers provide this advantage over built-in arrays.

11.3.2 **bitset**

Aspects of a system, such as the state of an input stream, are often represented as a set of flags indicating binary conditions such as good/bad, true/false, and on/off. C++ supports the notion of small sets of flags efficiently through bitwise operations on integers (§1.5). Class `bitset<N>` generalizes this notion and offers greater convenience by providing operations on a sequence of `N` bits `[0:N)`, where `N` is known at compile time. For sets of bits that don’t fit into a **long long int**, using a **bitset** is much more convenient than using integers directly. For smaller sets, **bitset** is usually optimized. If you want to name the bits, rather than numbering them, you can use a **set** (§9.4) or an enumeration (§2.5).

A **bitset** can be initialized with an integer or a string:

```
bitset<9> bs1 {"110001111"};
bitset<9> bs2 {399};
```

The usual bitwise operations (§1.5) can be applied, as can left- and right-shift operations (`<<` and `>>`):

```
bitset<9> bs3 = ~bs1;    // complement: bs3=="001110000"
bitset<9> bs4 = bs1&bs3; // all zeros
bitset<9> bs5 = bs1<<2;  // shift left: bs5 = "111000000"
```

The shift operators (here, `<<`) “shifts in” zeros.

The operations `to_ulong()` and `to_string()` provide the inverse operations to the constructors. For example, we could write out the binary representation of an **int**:

```
void binary(int i)
{
    bitset<8*sizeof(int)> b = i;    // assume 8-bit byte (see also §12.7)
    cout << b.to_string() << '\n'; // write out the bits of i
}
```


If you need more than two elements (or less), you can use **tuple** (from `<utility>`). A **tuple** is a heterogeneous sequence of elements; for example:

```
tuple<string,int,double> t2{"Sild",123, 3.14};    // the type is explicitly specified

auto t = make_tuple(string{"Herring"},10, 1.23); // the type is deduced to tuple<string,int,double>

string s = get<0>(t);    // get first element of tuple: "Herring"
int x = get<1>(t);       // 10
double d = get<2>(t);    // 1.23
```

The elements of a **tuple** are numbered (starting with zero), rather than named the way elements of **pairs** are (**first** and **second**). To get compile-time selection of elements, I must unfortunately use the ugly `get<1>(t)`, rather than `get(t,1)` or `t[1]`.

Like **pairs**, **tuples** can be assigned and compared if their elements can be.

A **pair** is common in interfaces because often we want to return more than one value, such as a result and an indicator of the quality of that result. It is less common to need three or more parts to a result, so **tuples** are more often found in the implementations of generic algorithms.

11.4 Time

The standard library provides facilities for dealing with time. For example, here is the basic way of timing something:

```
using namespace std::chrono;    // see §3.3

auto t0 = high_resolution_clock::now();
do_work();
auto t1 = high_resolution_clock::now();
cout << duration_cast<milliseconds>(t1-t0).count() << "msec\n";
```

The clock returns a **time_point** (a point in time). Subtracting two **time_points** gives a **duration** (a period of time). Various clocks give their results in various units of time (the clock I used measures **nanoseconds**), so it is usually a good idea to convert a **duration** into a known unit. That's what **duration_cast** does.

The standard-library facilities for dealing with time are found in the subnamespace **std::chrono** in `<chrono>`.

Don't make statements about "efficiency" of code without first doing time measurements. Guesses about performance are most unreliable.

11.5 Function Adaptors

A function adaptor takes a function as argument and returns a function object that can be used to invoke the original function. The standard library provides **bind()** and **mem_fn()** adaptors to do argument binding, also called *Currying* or *partial evaluation*. Binders were heavily used in the past, but most uses seem to be more easily expressed using lambdas (§5.5).

11.5.1 `bind()`

Given a function and a set of arguments, `bind()` produces a function object that can be called with “the remaining” arguments, if any, of the function. For example:

```
double cube(double);

auto cube2 = bind(cube,2);
```

A call `cube2()` will invoke `cube` with the argument `2`, that is, `cube(2)`. We don’t have to bind every argument of a function. For example:

```
using namespace placeholders;

void f(int,const string&);
auto g = bind(f,2,_1);           // bind f()'s first argument to 2
f(2,"hello");
g("hello");                     // also calls f(2,"hello");
```

The curious `_1` argument to the binder is a placeholder telling `bind()` where arguments to the resulting function object should go. In this case, `g()`’s (first) argument is used as `f()`’s second argument.

The placeholders are found in the (sub)namespace `std::placeholders` that is part of `<functional>`.

To bind arguments for an overloaded function, we have to explicitly state which version of the function we want to bind:

```
int pow(int,int);
double pow(double,double); // pow() is overloaded

auto pow2 = bind(pow,_1,2); // error: which pow()?
auto pow2 = bind((double(*)(double,double))pow,_1,2); // OK (but ugly)
```

I assigned the result of `bind()` to a variable declared using `auto`. This saves me the bother of specifying the return type of a call of `bind()`. That can be useful because the return type of `bind()` varies with the type of function to be called and the argument values stored. In particular, the returned function object is larger when it has to hold values of bound parameters. When we want to be specific about the types of the arguments required and the type of result returned, we can use a `function` (§11.5.3).

11.5.2 `mem_fn()`

The function adaptor `mem_fn(mf)` produces a function object that can be called as a nonmember function. For example:

```
void user(Shape* p)
{
    p->draw();
    auto draw = mem_fn(&Shape::draw);
    draw(p);
}
```

The major use of `mem_fn()` is when an algorithm requires an operation to be called as a nonmember function. For example:

```
void draw_all(vector<Shape*>& v)
{
    for_each(v.begin(),v.end(),mem_fn(&Shape::draw));
}
```

Thus, `mem_fn()` can be seen as a mapping from the object-oriented calling style to the functional one.

Often, lambdas provide a simple and general alternative to binders. For example:

```
void draw_all(vector<Shape*>& v)
{
    for_each(v.begin(),v.end(),[](Shape* p) { p->draw(); });
}
```

11.5.3 function

A `bind()` can be used directly, and it can be used to initialize an `auto` variable. In that, `bind()` resembles a lambda.

If we want to assign the result of `bind()` to a variable with a specific type, we can use the standard-library type `function`. A `function` is specified with a specific return type and a specific argument type. For example:

```
int f1(double);
function<int(double)> fct {f1}; // initialize to f1
int f2(int);

void user()
{
    fct = [](double d) { return round(d); }; // assign lambda to fct
    fct = f1;                               // assign function to fct
    fct = f2;                               // error: incorrect argument type
}
```

The standard-library `function` is a type that can hold any object you can invoke using the call operator `()`. That is, an object of type `function` is a function object (§5.5). For example:

```
int round(double x) { return static_cast<int>(floor(x+0.5)); } // conventional 4/5 rounding

function<int(double)> f; // f can hold anything that can be called with a double and return an int

enum class Round_style { truncate, round };

struct Round { // function object carrying a state
    Round_style s;
    Round(Round_style ss) :s(ss) { }
    int operator()(double x) const { return static_cast<int>((s==Round_style::round) ? (x+0.5) : x); };
};
```

I use `static_cast` (§14.2.3) to make it explicit that I want to return an `int`.

```

void t1()
{
    f = round;
    cout << f(7.6) << '\n';           // call through f to the function round

    f = Round(Round_style::truncate);
    cout << f(7.6) << '\n';           // call the function object

    Round_style style = Round_style::round;
    f = [style] (double x){ return static_cast<int>((style==Round_style::round) ? x+0.5 : x); };

    cout << f(7.6) << '\n';           // call the lambda

    vector<double> v {7.6};
    f = Round(Round_style::round);
    std::transform(v.begin(),v.end(),v.begin(),f);    // pass to algorithm

    cout << v[0] << '\n';             // transformed by the lambda
}

```

We get 8, 7, 8, and 8.

Obviously, **functions** are useful for callbacks, for passing operations as arguments, etc.

11.6 Type Functions

A *type function* is a function that is evaluated at compile-time given a type as its argument or returning a type. The standard library provides a variety of type functions to help library implementers and programmers in general to write code that take advantage of aspects of the language, the standard library, and code in general.

For numerical types, **numeric_limits** from `<limits>` presents a variety of useful information (§12.7). For example:

```
constexpr float min = numeric_limits<float>::min();    // smallest positive float
```

Similarly, object sizes can be found by the built-in **sizeof** operator (§1.5). For example:

```
constexpr int sz = sizeof(int);    // the number of bytes in an int
```

Such type functions are part of C++'s mechanisms for compile-time computation that allow tighter type checking and better performance than would otherwise have been possible. Use of such features is often called *metaprogramming* or (when templates are involved) *template metaprogramming*. Here, I just present two facilities provided by the standard library: **iterator_traits** (§11.6.1) and type predicates (§11.6.2).

11.6.1 iterator_traits

The standard-library **sort()** takes a pair of iterators supposed to define a sequence (Chapter 10). Furthermore, those iterators must offer random access to that sequence, that is, they must be

random-access iterators. Some containers, such as `forward_list`, do not offer that. In particular, a `forward_list` is a singly-linked list so subscripting would be expensive and there is no reasonable way to refer back to a previous element. However, like most containers, `forward_list` offers *forward iterators* that can be used to traverse the sequence by algorithms and `for`-statements (§5.2).

The standard library provides a mechanism, `iterator_traits` that allows us to check which kind of iterator is supported. Given that, we can improve the range `sort()` from §10.7 to accept either a `vector` or a `forward_list`. For example:

```
void test(vector<string>& v, forward_list<int>& lst)
{
    sort(v); // sort the vector
    sort(lst); // sort the singly-linked list
}
```

The techniques needed to make that work are generally useful.

First, I write two helper functions that take an extra argument indicating whether they are to be used for random-access iterators or forward iterators. The version taking random-access iterator arguments is trivial:

```
template<typename Ran> // for random-access iterators
void sort_helper(Ran beg, Ran end, random_access_iterator_tag) // we can subscript into [beg:end)
{
    sort(beg,end); // just sort it
}
```

The version for forward iterators simply copies the list into a `vector`, sorts, and copies back:

```
template<typename For> // for forward iterators
void sort_helper(For beg, For end, forward_iterator_tag) // we can traverse [beg:end)
{
    vector<Value_type<For>> v {beg,end}; // initialize a vector from [beg:end)
    sort(v.begin(),v.end());
    copy(v.begin(),v.end(),beg); // copy the elements back
}
```

`Value_type<For>>` is the type of `For`'s elements, called it's *value type*. Every standard-library iterator has a member `value_type`. I get the `Value_type<For>>` notation by defining a type alias (§5.7):

```
template<typename C>
using Value_type = typename C::value_type; // C's value type
```

Thus, `v` is a `vector<X>` where `X` is the element type of the input sequence.

The real “type magic” is in the selection of helper functions:

```
template<typename C>
void sort(C& c)
{
    using Iter = Iterator_type<C>;
    sort_helper(c.begin(),c.end(),Iterator_category<Iter>{});
}
```

Here, I use two type functions: `Iterator_type<C>` returns the iterator type of `C` (that is, `C::iterator`) and

then `iterator_category<Iter>{} constructs a “tag” value indicating the kind of iterator provided:`

- `std::random_access_iterator_tag` if `C`’s iterator supports random access.
- `std::forward_iterator_tag` if `C`’s iterator supports forward iteration.

Given that, we can select between the two sorting algorithms at compile time. This technique, called *tag dispatch* is one of several used in the standard library and elsewhere to improve flexibility and performance.

The standard-library support for techniques for using iterators, such as tag dispatch, comes in the form of a simple class template `iterator_traits` from `<iterator>`. This allows simple definitions of the type functions used in `sort()`:

```
template<typename C>
    using Iterator_type = typename C::iterator;    // C's iterator type

template<typename Iter>
    using Iterator_category = typename std::iterator_traits<Iter>::iterator_category; // Iter's category
```

If you don’t want to know what kind of “compile-time type magic” is used to provide the standard-library features, you are free to ignore facilities such as `iterator_traits`. But then you can’t use the techniques they support to improve your own code.

11.6.2 Type Predicates

A standard-library type predicate is a simple type function that answers a fundamental question about types. For example:

```
bool b1 = Is_arithmetic<int>();    // yes, int is an arithmetic type
bool b2 = Is_arithmetic<string>(); // no, std::string is not an arithmetic type
```

These predicates are found in `<type_traits>`. Other examples are `is_class`, `is_pod`, `is_literal_type`, `has_virtual_destructor`, and `is_base_of`. They are most useful when we write templates. For example:

```
template<typename Scalar>
class complex {
    Scalar re, im;
public:
    static_assert(Is_arithmetic<Scalar>(), "Sorry, I only support complex of arithmetic types");
    // ...
};
```

To improve readability compared to using the standard library directly, I defined a type function:

```
template<typename T>
constexpr bool Is_arithmetic()
{
    return std::is_arithmetic<T>::value ;
}
```

Older programs use `::value` directly instead of `()`, but I consider that quite ugly and it exposes implementation details.

11.7 Advice

- [1] The material in this chapter roughly corresponds to what is described in much greater detail in Chapters 33-35 of [Stroustrup,2013].
- [2] A library doesn't have to be large or complicated to be useful; §11.1.
- [3] A resource is anything that has to be acquired and (explicitly or implicitly) released; §11.2.
- [4] Use resource handles to manage resources (RAII); §11.2.
- [5] Use `unique_ptr` to refer to objects of polymorphic type; §11.2.1.
- [6] Use `shared_ptr` to refer to shared objects; §11.2.1.
- [7] Prefer resource handles with specific semantics to smart pointers; §11.2.1.
- [8] Prefer `unique_ptr` to `shared_ptr`; §4.6.4, §11.2.1.
- [9] Prefer smart pointers to garbage collection; §4.6.4, §11.2.1.
- [10] Use `array` where you need a sequence with a `constexpr` size; §11.3.1.
- [11] Prefer `array` over built-in arrays; §11.3.1.
- [12] Use `bitset` if you need `N` bits and `N` is not necessarily the number of bits in a built-in integer type; §11.3.2.
- [13] When using `pair`, consider `make_pair()` for type deduction; §11.3.3.
- [14] When using `tuple`, consider `make_tuple()` for type deduction; §11.3.3.
- [15] Time your programs before making claims about efficiency; §11.4.
- [16] Use `duration_cast` to report time measurements with proper units; §11.4.
- [17] Often, a lambda is an alternative to using `bind()` or `mem_fn()`; §11.5.
- [18] Use `bind()` to create variants of functions and function objects; §11.5.1.
- [19] Use `mem_fn()` to create function objects that can invoke a member function when called using the traditional function call notation; §11.5.2.
- [20] Use `function` when you need to store something that can be called; §11.5.3.
- [21] You can write code to explicitly depend on properties of types; §11.6.